

ForesightNav: Anticipatory Reinforcement Learning for UAV Navigation in Environments with Dynamic Obstacles and Large Non-Convex Structures

Abstract—Safe UAV navigation in environments with dynamic obstacles and large non-convex structures requires anticipating motion conflicts while avoiding local deadlocks caused by extended walls and concave geometry. However, many existing navigation methods either react primarily to instantaneous geometry or train policies using distance-based safety signals, which can result in late avoidance maneuvers and myopic decisions. To address these limitations, this paper presents ForesightNav, an anticipatory reinforcement-learning framework built on Proximal Policy Optimization that integrates finite-horizon motion-risk modeling with predictive outcome supervision. We introduce a finite-horizon, time-to-collision-aware three-dimensional velocity-obstacle reward that incorporates relative motion and obstacle geometry to encourage proactive avoidance of dynamic obstacles. Moreover, an action-conditioned multi-horizon auxiliary objective provides multi-step near-term outcome supervision by predicting task-relevant rollout outcomes following each observation–action pair. Its losses regularize the observation encoder shared by the actor and critic using semantically separated collision, deadlock, clearance, and progress signals. Extensive simulation experiments demonstrate that ForesightNav outperforms the compared state-of-the-art methods in dynamic-obstacle avoidance and navigation through environments containing large non-convex structures. Zero-shot real-world flights further validate its deployment feasibility without fine-tuning.

I. INTRODUCTION

AUTONOMOUS navigation is essential for unmanned aerial vehicles (UAVs) operating in environments with dynamic obstacles and large non-convex structures. Beyond reacting to local geometry, a UAV must account for the likely consequences of its decisions: an action that is collision-free at the current instant may still create a future motion conflict or steer the UAV into a local deadlock near an extended wall or within a concavity. Providing such foresight under limited onboard sensing and computation remains a central challenge in UAV navigation and control.

Classical navigation systems decompose the problem into perception, prediction, planning, and control [1]–[3]. Although interpretable and constraint-aware, they rely on accurate models, hand-crafted objectives, and repeated online optimization, which can increase tuning effort and reaction latency. Deep reinforcement learning offers an alternative by learning direct observation-to-action policies from large-scale simulated experience [4]–[6]. NavRL [6], for example, demonstrates the potential of this paradigm by combining Proximal Policy Optimization (PPO) [7], bounded three-dimensional velocity control, and efficient parallel training to produce responsive navigation commands through direct policy inference. Nevertheless, it leaves two limitations unresolved. Its training objective, like many local reactive formulations, emphasizes instantaneous clearance and goal progress: it neither explicitly models finite-horizon relative-motion conflicts nor directly

supervises whether a sequence of locally safe actions will circumnavigate an extended obstacle or escape a concavity [5, 6, 8, 9].

These limitations produce two recurring failure modes. First, instantaneous distance-based rewards can assign similar penalties to obstacles at similar distances, even though closing speed determines how quickly separation decreases and vertical alignment determines whether the three-dimensional safety volumes can overlap. These factors can yield substantially different times to collision, causing a rapidly approaching obstacle to elicit an adequate response only after the available avoidance window has narrowed. Second, locally collision-free actions can remain globally unproductive around large non-convex structures. Broad walls repeatedly obstruct the goal direction, while L- and U-shaped structures can trap a policy in oscillatory or stagnant behavior that appears safe at each individual step. Successful circumnavigation instead requires a temporally consistent bypass whose immediate goal progress may be small or temporarily negative. Predictive and model-based methods can provide temporal context or multi-step lookahead, but evaluating model rollouts online may increase deployment computation [10]–[12].

Motivated by these unresolved problems, we develop ForesightNav, an anticipatory reinforcement-learning framework that combines physics-informed finite-horizon collision-risk shaping with action-conditioned supervision of near-term navigation outcomes. The framework targets late responses to dynamic motion conflicts and deadlock-prone behavior around large non-convex structures while retaining efficient direct policy inference and avoiding online predictive rollout during action selection.

ForesightNav realizes this design through two mechanisms. First, it extends the classical velocity-obstacle (VO) concept [13] to a finite-horizon three-dimensional velocity-obstacle (3D-VO) test and maps the resulting time to collision (TTC) to a continuous risk signal. We refer to this training signal as the *TTC-aware 3D-VO reward*; unlike a conventional VO action filter, it shapes the nominal policy during learning. Second, an action-conditioned multi-horizon objective supervises task-relevant outcomes following each observation–action pair, including future collision and deadlock events, minimum forward clearance, and cumulative goal progress. The auxiliary losses train the observation encoder shared by the actor and critic but are omitted during deployment. Thus, the TTC-aware 3D-VO reward shapes return-based updates using analytic relative-motion risk, while multi-horizon supervision makes the shared representation predictive of realized near-term outcomes. The overall architecture and its training and deployment pathways are illustrated in Fig. 1.

We conduct comprehensive evaluations of ForesightNav

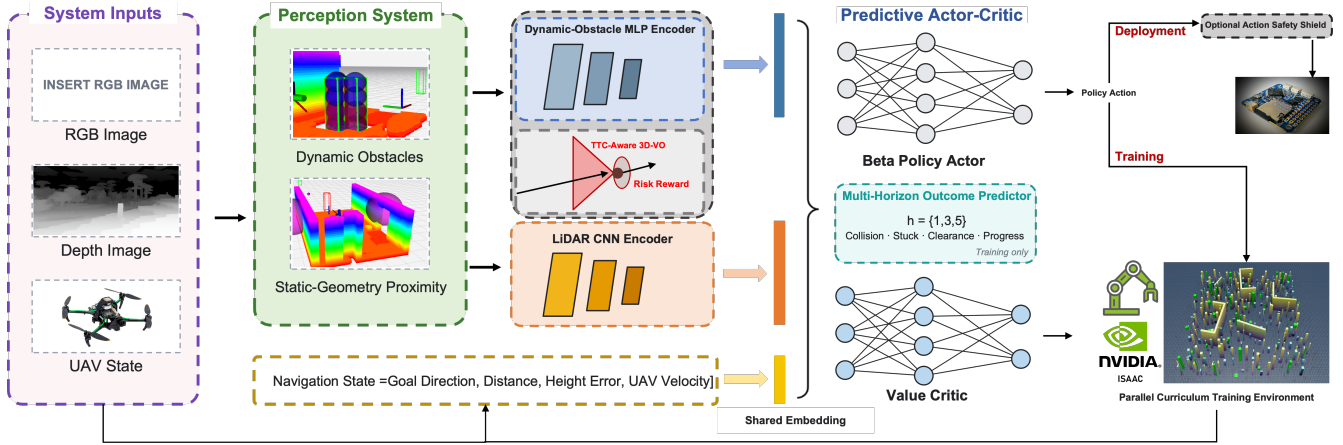


Fig. 1. The proposed ForesightNav framework. The perception system processes RGB-D observations together with the UAV state to form dynamic-obstacle, static-geometry proximity, and navigation representations. Modality-specific MLP and CNN encoders transform these inputs into a shared embedding for the predictive actor-critic. During training, the TTC-aware 3D-VO risk reward and action-conditioned multi-horizon outcome predictor jointly shape policy learning in parallel curriculum environments. During deployment, the outcome predictor is removed, and the Beta-policy actor directly generates velocity actions that can be further refined by an optional safety shield.

in simulation and real-world environments. In simulation, we compare it with NavRL, PE-Planner, and an ablation without the TTC-aware 3D-VO reward [1, 6] across static, dynamic, mixed, and large non-convex settings. ForesightNav and NavRL use identical deployment-time shield settings. The results show that ForesightNav outperforms the reference methods in dynamic-obstacle avoidance and is less susceptible to local trapping around large non-convex structures. Real-world UAV flights further validate its zero-shot deployment feasibility.

The main contributions of this work are summarized as follows:

- 1) **Physics-informed 3D dynamic-risk learning:** We introduce a time-to-collision-aware three-dimensional VO reward that provides finite-horizon dynamic-risk supervision and encourages proactive avoidance of dynamic obstacles.
- 2) **Action-conditioned predictive representation learning:** We introduce a multi-horizon auxiliary objective that provides task-oriented supervision for the shared actor-critic encoder during training.
- 3) **Evaluation under dynamic and non-convex geometric challenges:** Experiments demonstrate improved navigation under dynamic motion conflicts, stronger resistance to deadlocks around large non-convex structures, and zero-shot physical deployment feasibility.

II. RELATED WORK

A. Classical, Learning-Based, and Hybrid Navigation

Classical UAV navigation commonly combines path search with receding-horizon trajectory optimization [1]–[3]. FASTER maintains dynamically feasible trajectories in unknown environments [2], ViGO optimizes vision-aided B-spline trajectories around dynamic obstacles [3], and PE-Planner couples kinodynamic search and trajectory refinement with model predictive contouring control [1]. These methods provide explicit feasibility constraints and planning lookahead, but depend on sufficiently accurate environment models and

repeated online search or optimization, making their performance sensitive to perception quality and computation budget. Learning-based methods instead shift much of this computation to offline experience: CADRL learns an interaction-aware value function [4], Long *et al.* map range and goal observations directly to continuous controls [5], and NavRL combines three-dimensional velocity control with a deployment-time safety shield [6]. Direct policy inference is efficient, but scalar rewards provide only indirect and entangled supervision of distinct future collision, deadlock, clearance, and progress outcomes.

Large or non-convex structures expose a further distinction between immediate collision avoidance and successful obstacle circumnavigation. A global planner can reason about map connectivity and route around an extended boundary, but this capability requires adequate global coverage and repeated map updates and replanning, which are difficult to maintain in partially observed dynamic scenes. Local learning-based formulations avoid that dependency, yet typically represent only nearby geometry and optimize immediate clearance together with goal-directed motion [5, 6, 8]. They therefore provide limited guidance for maintaining a bypass direction along a broad wall or escaping an L- or U-shaped concavity, making locally safe deadlocks or local optima more likely. To address this issue, Lee *et al.* [9] use privileged time-of-arrival maps to guide quadrotors around large walls and dead ends.

Hybrid and safety-augmented methods combine learned policies with deployment-time planning, recovery, or action projection [14]–[18]. They can improve route reachability or enforce safety constraints, but introduce additional planner-policy interfaces, online computation, or dependence on maintained maps. Planner demonstrations and privileged expert trajectories can instead transfer structured guidance during training [19, 20], although their effectiveness depends on the coverage and quality of the expert data. These trade-offs motivate learning anticipatory behavior directly in the policy while retaining lightweight online inference.

B. Velocity-Obstacle-Guided Learning

The velocity-obstacle (VO) formulation identifies velocities that lead to collision under an assumed relative-motion model and supports online selection outside the resulting forbidden set [13]. Optimal Reciprocal Collision Avoidance (ORCA) represents reciprocal multi-agent avoidance through pairwise linear half-plane constraints and computes a feasible velocity close to the preferred command [21]. Such methods provide local kinematic collision avoidance online, but when used only as action filters, they correct a nominal command after it has been generated and do not teach the policy to avoid hazardous actions proactively.

VO structure has also been incorporated into reinforcement-learning objectives. DRL-VO uses a two-dimensional directional reward with LiDAR histories and pedestrian kinematics [8], while Han *et al.* [22] shape rewards using reciprocal velocity-obstacle (RVO) area and expected collision time for differential-drive robots. NavRL instead applies a VO-inspired filter after policy inference [6]. Unlike these planar reward formulations or post-policy filtering alone, ForesightNav maps finite-horizon collision times from a three-dimensional VO model to a training reward. The resulting signal accounts for obstacle extent, vertical separation, and relative motion, while the runtime shield remains a separate safeguard for residual unsafe actions.

C. Predictive Models and Auxiliary Outcome Learning

Predictive navigation methods exploit future structure in different ways. NavRep learns latent dynamics before policy optimization [10], RGL combines relational interaction modeling with multi-step crowd lookahead [11], and BADGR predicts action-conditioned events to evaluate candidate control sequences online [12]. Their predictions support representation pretraining or explicit lookahead, rather than directly supervising a shared actor-critic representation with separate task outcomes at multiple horizons. Auxiliary objectives provide a complementary approach: general value functions predict temporally extended signals [23], UNREAL combines reward prediction with auxiliary control and value replay [24], and SPR enforces action-conditioned multi-step latent consistency [25]. These objectives improve learned representations, but do not directly provide the combined collision, deadlock, clearance, and progress supervision required for the navigation failures considered here.

In contrast, ForesightNav jointly trains its shared policy representation using analytic finite-horizon motion risk and action-conditioned, horizon-specific navigation outcomes. This design targets late responses to dynamic motion conflicts and locally safe deadlocks around large non-convex structures, while the predictive branch is removed during deployment to preserve direct policy inference.

III. METHODOLOGY

In this section, we introduce the finite-horizon TTC-aware 3D-VO formulation, the reinforcement-learning problem, the predictive actor-critic, and the deployment-time shield. Here, “TTC-aware 3D-VO” denotes classical VO geometry extended

to three dimensions with an anisotropic envelope and finite horizon, whose earliest valid intersection time is mapped to a continuous risk.

A. Finite-Horizon TTC-Aware 3D Velocity-Obstacle Modeling

Instantaneous clearance cannot distinguish obstacles with similar separation but different relative motion. ForesightNav therefore extends the classical velocity-obstacle concept [13] to quantify finite-horizon 3-D motion conflicts. For the transition induced by $\mathbf{a}_t$, the model uses the resulting UAV position $\mathbf{p}_{t+1}$ and realized velocity $\mathbf{v}_{t+1}$, as illustrated in Fig. 2.

Obstacle i , with horizontal dimensions (w_i^x, w_i^y) , height h_i , center $\mathbf{p}_{i,t}^o$, and velocity $\mathbf{v}_{i,t}^o$, is represented by an overlapping vertical sphere stack. Its primitives have radius $b_i = \frac{1}{2}\sqrt{(w_i^x)^2 + (w_i^y)^2}$, spacing $l_i = \max(w_i^x, w_i^y)$, count $K_i = \lceil h_i/l_i \rceil$, and centers $\mathbf{p}_{i,j,t}^o = \mathbf{p}_{i,t}^o + [0, 0, (j - (K_i - 1)/2)l_i]^\top$, $j = 0, \dots, K_i - 1$. All inherit $\mathbf{v}_{i,t}^o$.

For prediction horizon $H > 0$, define the post-transition relative position and velocity of primitive (i, j) as $\mathbf{r} \equiv \mathbf{r}_{i,j,t+1} = \mathbf{p}_{i,j,t+1}^o - \mathbf{p}_{t+1}$ and $\mathbf{u} \equiv \mathbf{u}_{i,j,t+1} = \mathbf{v}_{i,t+1}^o - \mathbf{v}_{t+1}$, respectively. The primitive is retained only if the interval between $r_{i,j,t+1}^z$ and $r_{i,j,t+1}^z + H u_{i,j,t+1}^z$ intersects $[-(b_i + m_h), b_i + m_h]$, where the superscript z selects the vertical component. Margins m_{xy} and m_z encode the horizontal and vertical UAV envelope plus safety buffers, and $m_h \geq m_z$ is a conservative vertical prefilter margin. For each retained primitive, define

$$\begin{aligned} \mathbf{D}_{i,j} &= \text{diag}(b_i + m_{xy}, b_i + m_{xy}, b_i + m_z), \\ \bar{\mathbf{r}} &= \mathbf{D}_{i,j}^{-1} \mathbf{r}, \quad \bar{\mathbf{u}} = \mathbf{D}_{i,j}^{-1} \mathbf{u}. \end{aligned} \quad (1)$$

Under constant relative velocity, the finite-horizon velocity obstacle is

$$\mathcal{VO}_{i,j,t+1}^H = \left\{ \mathbf{v} \in \mathbb{R}^3 \mid \exists s \in (0, H] : \left\| \mathbf{D}_{i,j}^{-1} [\mathbf{r} + s(\mathbf{v}_{i,t+1}^o - \mathbf{v})] \right\|_2 \leq 1 \right\}. \quad (2)$$

ForesightNav tests the realized velocity $\mathbf{v}_{t+1}$ for membership rather than constructing the full set. The boundary-intersection condition is

$$\begin{aligned} As^2 + Bs + C &= 0, \\ A &= \bar{\mathbf{u}}^\top \bar{\mathbf{u}}, \quad B = 2\bar{\mathbf{r}}^\top \bar{\mathbf{u}}, \quad C = \bar{\mathbf{r}}^\top \bar{\mathbf{r}} - 1. \end{aligned} \quad (3)$$

For a non-overlapping primitive, a valid time to collision exists when relative motion is nonzero and approaching, the discriminant $\Delta = B^2 - 4AC$ is nonnegative, and

$$T_{i,j,t+1} = \frac{-B - \sqrt{\Delta}}{2A} \in (0, H]. \quad (4)$$

A primitive with $C \leq 0$ is already inside the inflated region and receives unit risk; otherwise, risk decays exponentially with TTC using decay constant $\tau > 0$:

$$\rho_{i,j,t+1} = \begin{cases} 1, & C \leq 0, \\ \exp(-T_{i,j,t+1}/\tau), & T_{i,j,t+1} \in (0, H], \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

The overall post-transition risk is $\rho_{t+1}^{\text{VO}} = \max_{i,j} \rho_{i,j,t+1}$ over retained primitives. The maximum emphasizes the most imminent conflict, keeps the risk bounded in $[0, 1]$, and prevents

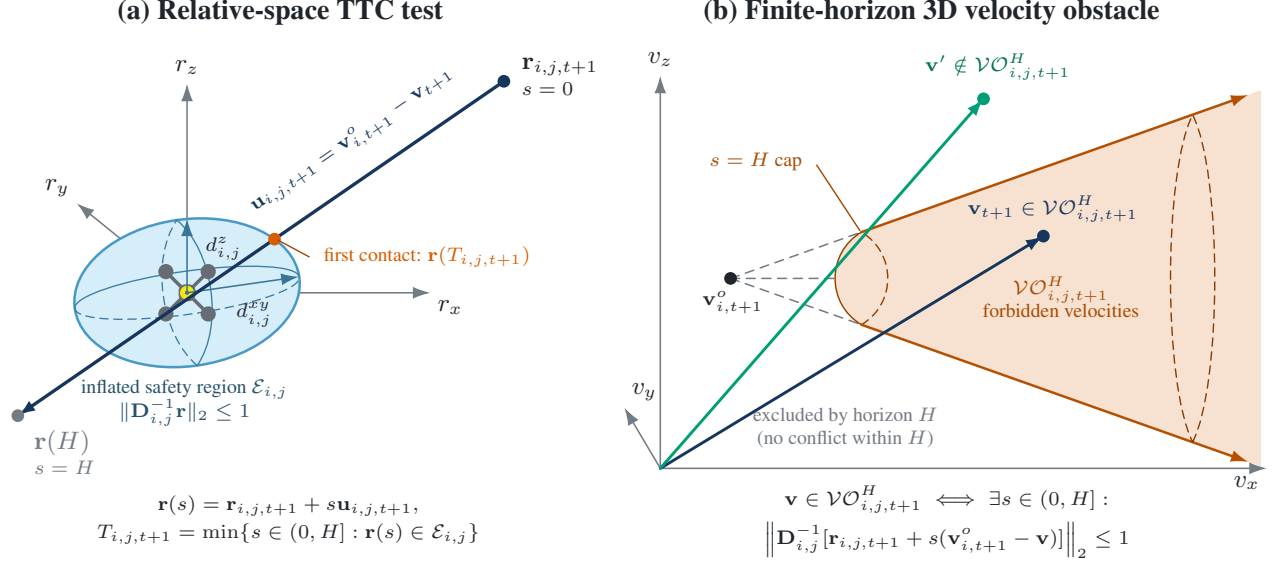


Fig. 2. Finite-horizon TTC-aware 3D-VO geometry for one retained primitive. (a) Relative motion first intersects the inflated region at $T_{i,j,t+1}$. (b) The corresponding realized velocity lies inside the finite-horizon VO; the $s = H$ cap excludes conflicts beyond the prediction horizon.

its magnitude from scaling with the number of primitives. This training-time measure assumes constant relative velocity over the finite horizon and does not constitute a formal safety guarantee.

B. Reinforcement-Learning Formulation

1) *Problem Formulation*: ForesightNav formulates local UAV navigation as a partially observable Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \Omega, O, \gamma)$, where $\mathcal{S}$ is the latent environment-state space, $\mathcal{A}$ the velocity-action space, P the transition model, R the reward function, Ω the observation space, O the observation model, and $\gamma \in [0, 1)$ the discount factor. At time t , the policy receives $\mathbf{o}_t \sim O(\cdot | \mathbf{x}_t)$, samples $\mathbf{a}_t \sim \pi_\theta(\cdot | \mathbf{o}_t)$, and induces

$$\mathbf{x}_{t+1} \sim P(\cdot | \mathbf{x}_t, \mathbf{a}_t), \quad r_t = R(\mathbf{x}_t, \mathbf{a}_t, \mathbf{x}_{t+1}). \quad (6)$$

The objective over an episode of at most T transitions is

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right]. \quad (7)$$

Here, $\mathbf{x}_t \in \mathcal{S}$, $\mathbf{o}_t \in \Omega$, and $\mathbf{a}_t \in \mathcal{A}$ denote the latent state, local observation, and action, respectively. Reward and auxiliary targets may depend on latent variables excluded from the policy observation. At the beginning of each episode, a fixed navigation frame G is defined with its origin at the initial UAV position, its horizontal x -axis pointing from the initial position to the goal, and its z -axis aligned with the world vertical.

2) *State and Observation Space*: A compact local observation compatible with the navigation interface in [6] combines static proximity, navigation state, and tracked dynamic obstacles:

$$\mathbf{o}_t = (\mathbf{L}_t, \mathbf{s}_t, \mathbf{D}_t), \quad \mathbf{L}_t \in \mathbb{R}^{36 \times 4}, \quad \mathbf{s}_t \in \mathbb{R}^8, \quad \mathbf{D}_t \in \mathbb{R}^{5 \times 10}. \quad (8)$$

Here, $\mathbf{L}_t$ contains 144 yaw-aligned proximity values arranged over 36 azimuths and four elevations, and

$$\mathbf{s}_t = [\hat{\mathbf{r}}_{g,t}^G, d_{g,t}^{xy}, \Delta z_{g,t}, \mathbf{v}_t^G], \quad (9)$$

where $\hat{\mathbf{r}}_{g,t}^G$ is the unit vector from the UAV to the goal, $d_{g,t}^{xy}$ is the horizontal goal distance, $\Delta z_{g,t}$ is the signed vertical goal displacement, and $\mathbf{v}_t^G$ is the UAV velocity, expressed in G where applicable. The matrix $\mathbf{D}_t$ stacks descriptors for the five nearest in-range dynamic obstacles, with unused rows zero padded. Each valid row is

$$\mathbf{d}_{i,t} = [\hat{\mathbf{r}}_{i,t}^G, d_{i,t}^{xy}, \Delta z_{i,t}, \mathbf{v}_{i,t}^{o,G}, q_w(w_i), q_h(h_i)] \in \mathbb{R}^{10}, \quad (10)$$

where $\hat{\mathbf{r}}_{i,t}^G$ is the unit relative-position vector, $d_{i,t}^{xy}$ the horizontal center distance, $\Delta z_{i,t}$ the signed vertical displacement, and $\mathbf{v}_{i,t}^{o,G}$ the obstacle velocity. The size codes follow the implementation: $q_w(w_i) = \text{clip}(w_i/\delta_w - 1, 0, 3)$, with $\delta_w = 0.25$ m, and $q_h(h_i) = h_i$ for finite-height obstacles with $h_i \leq h_{th} = 1.0$ m. Vertically spanning obstacles use $q_h(h_i) = 0$ and $\Delta z_{i,t} = 0$, distinguishing them from finite-height obstacles through this joint encoding.

3) *Action Space*: To obtain bounded commands without post-sampling clipping, the actor uses a factorized Beta policy [6]. It produces positive shape parameters α_t and β_t , samples the normalized action $\tilde{\mathbf{a}}_t$, and rescales it to a three-dimensional velocity command in G :

$$\tilde{\mathbf{a}}_t \sim \text{Beta}(\alpha_t, \beta_t), \quad \mathbf{a}_t^G = v_{\text{lim}}(2\tilde{\mathbf{a}}_t - \mathbf{1}) \in [-v_{\text{lim}}, v_{\text{lim}}]^3. \quad (11)$$

The limit is $v_{\text{lim}} = 2.0$ m/s. The command is rotated into the world frame and tracked by a low-level controller; yaw is regulated separately.

C. Reward Function

The reward combines five principal components that jointly promote anticipatory dynamic-obstacle avoidance,

goal-directed motion, smooth control, static clearance, and altitude regulation:

$$r_t^{\text{shape}} = \lambda_{\text{VO}} r_t^{\text{VO}} + \lambda_g r_t^{\text{goal}} + \lambda_{\text{sm}} r_t^{\text{sm}} + \lambda_{\text{ss}} r_t^{\text{ss}} + \lambda_h r_t^h. \quad (12)$$

where the nonnegative coefficients λ_{VO} , λ_g , λ_{sm} , λ_{ss} , and λ_h weight the respective terms. This combination distinguishes imminent motion conflicts from geometric proximity and balances safety with efficient, physically regular navigation.

1) *TTC-Aware 3D-VO Risk Reward*: The finite-horizon risk in Sec. III-A enters the reward as

$$r_t^{\text{VO}} = -\eta(n) \rho_{t+1}^{\text{VO}}, \quad \eta(n) = \min\left(\frac{n}{N_{\text{warm}}}, 1\right), \quad (13)$$

where n is the optimization step and N_{warm} is the warmup duration in optimization steps. The warmup factor gradually introduces anticipatory collision-risk shaping and prevents it from dominating before basic goal-directed behavior is established.

2) *Goal-Progress Reward*: Let $d_t = \|\mathbf{p}_g - \mathbf{p}_t\|_2$. The signed progress over the transition is

$$r_t^{\text{goal}} = d_t - d_{t+1}. \quad (14)$$

Approach to the goal is rewarded, while retreat is penalized with equal magnitude. Consequently, a closed forward-backward cycle cannot obtain positive undiscounted progress reward through asymmetric shaping.

3) *Smoothness Reward*: Abrupt changes in realized velocity are discouraged by

$$r_t^{\text{sm}} = -\|\mathbf{v}_{t+1} - \mathbf{v}_t\|_2. \quad (15)$$

This term favors temporally consistent motion and suppresses oscillatory control during local avoidance.

4) *Static-Safety Reward*: Let $d_{k,t+1}^L$ be the range of LiDAR ray k , N_L the number of rays, and m_s a safety margin. Static proximity is penalized by

$$r_t^{\text{ss}} = -\frac{1}{N_L} \sum_{k=1}^{N_L} \frac{\max(m_s - d_{k,t+1}^L, 0)}{m_s}. \quad (16)$$

Only geometry inside the prescribed margin contributes, while averaging keeps the reward scale independent of LiDAR resolution.

5) *Height Reward*: To discourage unnecessary vertical detours, altitude deviation beyond a tolerance ϵ_z is penalized quadratically:

$$r_t^h = -\max(|z_{t+1} - z_g| - \epsilon_z, 0)^2. \quad (17)$$

The tolerance permits necessary altitude adjustments near obstacles, while the quadratic growth prevents avoidance through excessive ascent or descent.

D. Network Design and Policy Training

The architecture uses modality-specific LiDAR and dynamic-obstacle encoders [6] and fuses their outputs with the navigation state. The resulting shared encoder E_ω maps $\mathbf{o}_t$ to the predictive representation $\mathbf{z}_t = E_\omega(\mathbf{o}_t)$. The actor π_θ generates the bounded Beta policy in Sec. III-B, while the critic estimates the observation-conditioned value $V_\psi(\mathbf{o}_t) = V_\psi(\mathbf{z}_t)$;

it does not receive the latent environment state. The actor, critic, and outcome-learning pathway are optimized around this common representation.

To make $\mathbf{z}_t$ informative about the consequences of the selected action, ForesightNav introduces multi-horizon action-conditioned outcome learning. For each horizon $h \in \mathcal{H} = \{1, 3, 5\}$, a lightweight decoder predicts

$$\hat{\mathbf{y}}_t^{(h)} = g_\phi^{(h)}(\mathbf{z}_t, \tilde{\mathbf{a}}_t) = [\hat{c}_t^{(h)}, \hat{b}_{\text{stuck},t}^{(h)}, \hat{f}_t^{(h)}, \hat{p}_t^{(h)}], \quad (18)$$

where $\hat{c}_t^{(h)}$ and $\hat{b}_{\text{stuck},t}^{(h)}$ are collision and persistent-stuck logits, and $\hat{f}_t^{(h)}$ and $\hat{p}_t^{(h)}$ predict minimum forward clearance and cumulative goal progress. Targets are obtained from the collected transition sequence:

$$\begin{aligned} c_t^{(h)} &= \max_{1 \leq k \leq h} c_{t+k}, & b_{\text{stuck},t}^{(h)} &= \max_{1 \leq k \leq h} b_{\text{stuck},t+k}, \\ f_t^{(h)} &= \min_{1 \leq k \leq h} f_{t+k}, & p_t^{(h)} &= \sum_{k=1}^h \Delta d_{t+k}, \end{aligned} \quad (19)$$

where c_t and $b_{\text{stuck},t}$ are the per-step binary collision and persistent-stuck indicators, f_t is the forward clearance, and $\Delta d_t = d_{t-1} - d_t$ is one-step goal progress. All aggregations are truncated at episode termination. Only the current action is supplied explicitly; later actions follow the rollout policy. Hence, multi-step outputs represent outcomes of the current action under policy continuation rather than under a prescribed action sequence.

For compactness, let $\tilde{f}_t^{(h)} = \text{symlog}(f_t^{(h)})$ and $\tilde{p}_t^{(h)} = \text{symlog}(p_t^{(h)})$, where $\text{symlog}(x) = \text{sign}(x) \log(1 + |x|)$. The predictive objective averages the four tasks over all horizons:

$$\begin{aligned} \mathcal{L}_{\text{MH}} &= \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} [\ell_{\text{BCE}}(\hat{c}_t^{(h)}, c_t^{(h)}) \\ &\quad + \ell_{\text{BCE}}(\hat{b}_{\text{stuck},t}^{(h)}, b_{\text{stuck},t}^{(h)}) \\ &\quad + \ell_{\text{SL1}}(\hat{f}_t^{(h)}, \tilde{f}_t^{(h)}) \\ &\quad + \ell_{\text{SL1}}(\hat{p}_t^{(h)}, \tilde{p}_t^{(h)})]. \end{aligned} \quad (20)$$

Policy and outcome learning are integrated through

$$\mathcal{L} = \mathcal{L}_{\text{PPO}} + \lambda_{\text{MH}} \mathcal{L}_{\text{MH}}, \quad \lambda_{\text{MH}} = 0.1. \quad (21)$$

The sampled action and rollout targets are treated as fixed inputs; $\text{sg}(\cdot)$ in Algorithms 1 and 2 denotes the stop-gradient operator used for the sampled action. Thus, $\mathcal{L}_{\text{MH}}$ updates the outcome decoder and shared encoders but has no direct gradient path to the actor or critic output heads. PPO makes $\mathbf{z}_t$ useful for action selection and value estimation, while outcome supervision simultaneously makes it predictive of near-term collision, trapping, clearance, and progress.

This predictive actor-critic representation is neither recurrent nor a generative world model: it reconstructs no future observation and performs no latent rollout for action selection. The decoder is omitted during execution, while its representation-level supervision remains embedded in the shared encoder, adding no online inference cost. PPO uses clipped policy and value objectives, generalized advantage estimation, and entropy regularization [7]. A stage-wise curriculum progressively increases dynamic-interaction density

Algorithm 1: TTC-Aware Predictive PPO Training

Input: stages $\{\mathcal{E}_q\}$, horizons $\mathcal{H} = \{1, 3, 5\}$, λ_{MH}

- 1 Initialize E_ω , π_θ , V_ψ , and g_ϕ
- 2 **foreach** curriculum stage $\mathcal{E}_q$ **do**
- 3 Load the selected stage- $(q-1)$ checkpoint if $q > 1$
- 4 **while** stage budget remains **do**
- 5 **foreach** parallel rollout step t **do**
- 6 $\mathbf{z}_t \leftarrow E_\omega(\mathbf{o}_t)$, $\tilde{\mathbf{a}}_t \sim \pi_\theta(\cdot | \mathbf{z}_t)$
- 7 Execute the scaled action and observe the transition
- 8 Evaluate ρ_{t+1}^{VO} and add $-\lambda_{\text{VO}}\eta(n)\rho_{t+1}^{\text{VO}}$ to r_t
- 9 Store the transition and $(c, b_{\text{stuck}}, f, \Delta d)_{t+1}$
- 10 Compute GAE and $\mathcal{L}_{\text{PPO}}$
- 11 **foreach** t and $h \in \mathcal{H}$ **do**
- 12 Build terminal-truncated target
- 13 $\mathbf{y}_t^{(h)} = (\max c, \max b_{\text{stuck}}, \min f, \sum \Delta d)$
- 14 $\hat{\mathbf{y}}_t^{(h)} \leftarrow g_\phi^{(h)}(\mathbf{z}_t, \text{sg}(\tilde{\mathbf{a}}_t))$
- 15 Compute $\mathcal{L}_{\text{MH}}$ (BCE for events; Smooth-L1 for continuous outcomes)
- 16 Update (θ, ψ) with $\mathcal{L}_{\text{PPO}}$, ϕ with $\mathcal{L}_{\text{MH}}$, and ω with $\mathcal{L}_{\text{PPO}} + \lambda_{\text{MH}}\mathcal{L}_{\text{MH}}$

Output: trained encoder, Beta-policy actor, and critic

Algorithm 2: Multi-Horizon Outcome Learning

Input: rollout outcomes $\{c_k, b_{\text{stuck},k}, f_k, \Delta d_k\}$, features $\mathbf{z}_t$, actions $\tilde{\mathbf{a}}_t$, $\mathcal{H} = \{1, 3, 5\}$

- 1 $\mathcal{L}_{\text{MH}} \leftarrow 0$, $N \leftarrow 0$
- 2 **foreach** valid rollout index t **do**
- 3 **foreach** $h \in \mathcal{H}$ **do**
- 4 $\bar{h} \leftarrow \min(h, \text{steps remaining before termination})$
- 5 $c_t^{(h)} \leftarrow \max_{1 \leq k \leq \bar{h}} c_{t+k}, b_{\text{stuck},t}^{(h)} \leftarrow \max_{1 \leq k \leq \bar{h}} b_{\text{stuck},t+k}$
- 6 $f_t^{(h)} \leftarrow \min_{1 \leq k \leq \bar{h}} f_{t+k}, p_t^{(h)} \leftarrow \sum_{k=1}^{\bar{h}} \Delta d_{t+k}$
- 7 $\hat{\mathbf{y}}_t^{(h)} \leftarrow g_\phi^{(h)}(\mathbf{z}_t, \text{sg}(\tilde{\mathbf{a}}_t))$
- 8 $\ell_t^{(h)} \leftarrow \ell_{\text{BCE}}(\hat{c}, c) + \ell_{\text{BCE}}(\hat{b}_{\text{stuck}}, b_{\text{stuck}})$
- 9 $\ell_t^{(h)} \leftarrow \ell_t^{(h)} + \ell_{\text{SL1}}(\hat{f}, \text{symlog}(f)) + \ell_{\text{SL1}}(\hat{p}, \text{symlog}(p))$
- 10 $\mathcal{L}_{\text{MH}} \leftarrow \mathcal{L}_{\text{MH}} + \ell_t^{(h)}, N \leftarrow N + 1$

Output: $\mathcal{L}_{\text{MH}}/N$ for updating g_ϕ and the shared encoder E_ω

and introduces more challenging large non-convex structures, strengthening both dynamic-obstacle avoidance and navigation around confining geometry while retaining previously acquired behavior.

E. Deployment-Time Policy Action Safety Shield

An optional VO-based action shield [6] provides a final corrective layer during deployment. It passes through policy velocities that satisfy the active collision-avoidance and control constraints and minimally adjusts commands predicted to be unsafe. The shield operates only after policy inference and is excluded from optimization. It complements, rather than replaces, the training-time finite-horizon TTC-aware 3D-VO objective: the learned policy supplies anticipatory avoidance, while the shield handles residual unsafe actions.

IV. EXPERIMENT RESULTS

In this section, we compare the proposed method with several state-of-the-art approaches in simulation environments

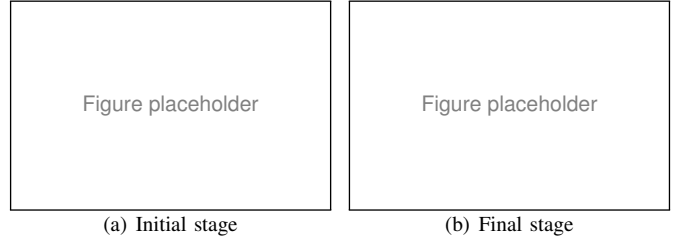


Fig. 3. Visualization of the curriculum-training environments in a $50 \text{ m} \times 50 \text{ m}$ workspace. Dynamic-obstacle density is progressively increased during training. (a) The initial environment contains 60 dynamic obstacles. (b) The final environment contains multiple large non-convex structures.

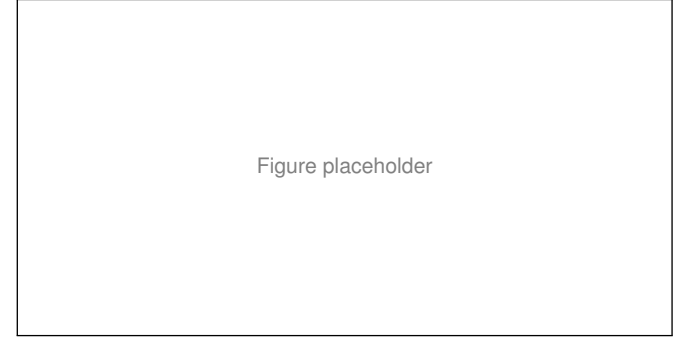


Fig. 4. Navigation success-rate curves over the five curriculum-training stages.

containing static obstacles, dynamic obstacles, mixed obstacles, and large non-convex structures. We further conduct real-world experiments to validate the effectiveness and generalizability of the proposed method.

A. Training Details and Setup

The policy is trained in NVIDIA Isaac Sim using PyTorch. The training scene covers a $50 \text{ m} \times 50 \text{ m}$ horizontal workspace with geometry represented up to a height of 4.5 m. The maximum UAV speed is 2.0 m/s. To expose the policy to different obstacle scales, the radii of cylindrical dynamic obstacles are set to $\{0.125, 0.25, 0.375, 0.50\} \text{ m}$, while their speeds are sampled from 0.5 to 1.5 m/s. Training is parallelized over 1,024 UAV instances with independently randomized navigation tasks on an NVIDIA GeForce RTX 4090 GPU, and each curriculum stage requires approximately 12 hours. Fig. 3 illustrates the progression from the initial dynamic-obstacle setting to the final environment containing large non-convex structures. The corresponding success-rate curves over the five curriculum stages are reported in Fig. 4. In addition, the hyperparameters used for training are listed in Table I; they were selected and tuned based on preliminary experiments to ensure stable and efficient training.

For quantitative deployment evaluation, we conduct 1,000 trials for each method in each scenario. A new start-goal pair is sampled for every trial, while obstacles evolve according to the motion process defined by the evaluated scenario. The environment configuration and metric thresholds are held fixed. A trial terminates upon goal arrival, collision, or the 180 s time limit. Let $N_{\text{trial}} = 1,000$ be the number of trials, and let N_{succ} , N_{coll} , and N_{timeout} denote the numbers of successful, collided, and timed-out trials, respectively. We further denote by $N_{\text{dead}}^{\text{evt}}$ the total number of deadlock events

TABLE I
KEY HYPERPARAMETERS USED FOR POLICY TRAINING

Parameter	Value	Parameter	Value
Parallel UAVs	1,024	Rollout length	32
PPO epochs	4	Minibatches	16
Discount factor γ	0.99	GAE parameter λ_{GAE}	0.95
Initial LRs (en-coder/actor/critic)	$(5, 5, 5) \times 10^{-4}$	Adaptation LRs (en-coder/actor/critic)	$10^{-5}, 10^{-5}, 10^{-4}$
Prediction horizons $\mathcal{H}$	$\{1, 3, 5\}$	Outcome-loss weight λ_{MH}	0.1
VO horizon H_{VO}	2.1 s	TTC decay τ	0.75 s
VO reward weight λ_{VO}	1.3	VO warmup	20,000 steps

detected across all trials and by $N_{\text{rec}}^{\text{evt}}$ the recovered-deadlock-event count. We report the following metrics:

- **Success Rate** ($N_{\text{succ}}/N_{\text{trial}}$): The fraction of trials in which the UAV reaches the goal before collision or timeout.
- **Collision Rate** ($N_{\text{coll}}/N_{\text{trial}}$): The fraction of trials terminated by contact with a static or dynamic obstacle.
- **Timeout Rate** ($N_{\text{timeout}}/N_{\text{trial}}$): The fraction of trials that neither reach the goal nor collide within 180 s.
- **Deadlock Count** ($N_{\text{dead}}^{\text{evt}}$): The total number of deadlock events detected over the 1,000 trials. A deadlock is registered when a forward obstacle co-occurs with goal progress no greater than 0.005 m for 40 consecutive evaluation cycles (2 s at 20 Hz). Because one trial may contain multiple events, this count is not bounded by N_{trial} .
- **Post-Deadlock Escape Rate** ($N_{\text{rec}}^{\text{evt}}/N_{\text{dead}}^{\text{evt}}$): The fraction of detected deadlock events that satisfy the recovery criterion. This event-level metric is distinct from the fraction of deadlocked trials that eventually reach the goal and is reported as undefined when no deadlock event occurs.

Taken together, these metrics characterize the UAV’s ability to avoid dynamic obstacles and navigate around large non-convex structures, including its tendency to avoid or recover from deadlocks and local optima. Comparisons with the corresponding ablations further assess whether TTC-aware and multi-horizon prediction enables earlier responses to evolving motion conflicts.

B. Simulation Results

We conducted simulation experiments in Gazebo to evaluate ForesightNav and its sim-to-sim transferability from the training environment. The quantitative evaluation covers four challenging settings containing static obstacles, dynamic obstacles, mixed obstacles, and large non-convex structures. We compare ForesightNav with the learning-based NavRL policy [6], the optimization-based PE-Planner [1], and an ablation without the TTC-aware 3D-VO reward. Each method is evaluated over 1,000 independent trials under the same workspace, start-goal sampling, obstacle configuration, and termination criteria. Fig. 5(a)–(c) shows representative trajectories generated by

ForesightNav through static, dynamic, and mixed obstacle fields, while Fig. 5(d) illustrates its sustained detour around large non-convex structures. Fig. 6 further provides a detailed ForesightNav trajectory in a corridor containing both static obstacles and moving pedestrians. The quantitative results are reported in Table II.

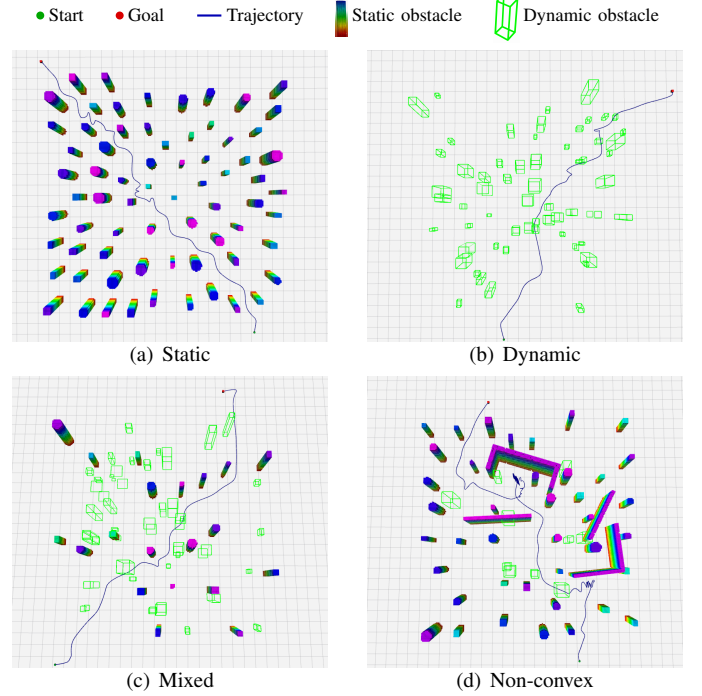


Fig. 5. Representative ForesightNav trajectories in the four simulation environments.

ForesightNav maintains performance comparable to NavRL in the static environment and provides a clearer advantage when obstacle motion must be anticipated. In the purely dynamic environment, its success rate reaches 83.3%, compared with 69.9% for NavRL. The same overall trend is observed in the mixed environment, where ForesightNav achieves the highest success rate and the lowest collision rate. The degradation caused by removing the 3D-VO reward further indicates that finite-horizon relative-motion supervision improves responses to evolving motion conflicts rather than merely increasing static clearance.

In the environment containing large non-convex structures, both ForesightNav variants reduce timeouts and deadlock-related failures relative to NavRL and PE-Planner, while recovering a larger proportion of detected deadlocks. The complete method and the 3D-VO ablation exhibit mixed differences on individual metrics; therefore, this environment demonstrates the benefit of the broader predictive training design under confining geometry rather than an isolated advantage of the 3D-VO component. Overall, the results show that ForesightNav preserves static-obstacle navigation while improving anticipatory dynamic avoidance and robustness to local trapping around large structures.

To isolate the contribution of deployment-time action correction, Table III compares NavRL and ForesightNav with

TABLE II

QUANTITATIVE COMPARISON OVER 1,000 TRIALS IN FOUR OBSTACLE ENVIRONMENTS WITHIN A $20\text{ m} \times 20\text{ m}$ WORKSPACE. THE BEST NON-TIED RESULT IN EACH ENVIRONMENT IS SHOWN IN BOLD.

Environment	Method	Success Rate (%) $\uparrow$	Collision Rate (%) $\downarrow$	Timeout Rate (%) $\downarrow$	Deadlock Count $\downarrow$	Post-Deadlock Escape Rate (%) $\uparrow$
Static obstacles ($N_{\text{stat}} = 70$)	PE-Planner	85.5	14.5	0	0	N/A
	NavRL	94.1	5.9	0	0	N/A
	ForesightNav w/o 3D-VO	92.7	7.3	0	0	N/A
	ForesightNav	94.4	5.6	0	0	N/A
Dynamic obstacles ($N_{\text{dyn}} = 60$)	PE-Planner	65.6	34.4	0	0	N/A
	NavRL	69.9	30.1	0	0	N/A
	ForesightNav w/o 3D-VO	71.2	28.8	0	0	N/A
	ForesightNav	83.3	16.7	0	0	N/A
Mixed static and dynamic obstacles ($N_{\text{stat}} = 20$) ($N_{\text{dyn}} = 40$)	PE-Planner	63.2	36.8	0	0	N/A
	NavRL	61.8	38.2	0	0	N/A
	ForesightNav w/o 3D-VO	60.3	39.7	0	0	N/A
	ForesightNav	76.5	23.5	0	0	N/A
Static obstacles and large non-convex structures	PE-Planner	58.9	15.9	25.2	127	21/127 (16.5%)
	NavRL	66.7	13.6	19.7	93	23/93 (24.7%)
	ForesightNav w/o 3D-VO	77.5	14.0	8.5	42	18/42 (42.9%)
	ForesightNav	78.1	15.6	6.3	53	24/53 (45.3%)

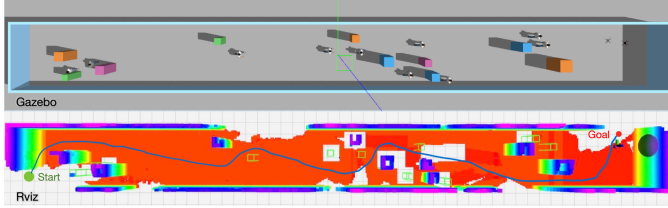


Fig. 6. Safe navigation in a simulated corridor environment. The upper panel shows the Gazebo scene containing static obstacles and moving pedestrians, while the lower panel shows the corresponding environment map and UAV trajectory.

TABLE III

EFFECT OF THE DEPLOYMENT-TIME SAFETY SHIELD OVER 1,000 TRIALS IN THE MIXED STATIC AND DYNAMIC OBSTACLE ENVIRONMENT WITHIN A $20\text{ m} \times 20\text{ m}$ WORKSPACE.

Method	Shield	Success Rate (%) $\uparrow$	Collision Rate (%) $\downarrow$	Timeout Rate (%) $\downarrow$
NavRL	Off	57.4	42.6	N/A
	On	62.7	37.3	N/A
ForesightNav	Off	64.2	35.8	N/A
	On	75.3	24.7	N/A

and without the safety shield in the mixed static–dynamic environment.

Enabling the shield consistently increases the success rate and reduces collisions for both NavRL and ForesightNav, with a larger improvement observed for ForesightNav. This result confirms that learned anticipatory avoidance and online action correction are complementary: the policy provides the nominal navigation behavior, while the shield mitigates residual unsafe

commands.

C. Physical Flight Tests

To evaluate zero-shot sim-to-real transfer, we deploy the policy selected in simulation on a quadrotor without additional fine-tuning. The experiments are conducted in a $28\text{ m} \times 15\text{ m}$ flight area, and the maximum translational velocity is set to 1.2 m/s . The onboard navigation stack supplies the policy with the same local observation schema used during training, comprising proximity measurements, the estimated UAV navigation state, and tracked dynamic-obstacle descriptors. ForesightNav produces three-dimensional velocity commands that are tracked by the low-level flight controller. The multi-horizon outcome predictor and TTC-aware 3D-VO risk computation are used only for training and are not evaluated onboard.

We construct four physical environments to examine complementary navigation capabilities. As shown in Fig. 7(a)–(d), the first environment contains static obstacles, the second introduces moving obstacles, the third combines static and moving obstacles, and the fourth contains large non-convex structures. These settings respectively examine local clearance maintenance, response to relative obstacle motion, avoidance under simultaneous geometric and motion constraints, and sustained detours around structures that can induce local trapping. The four layouts therefore provide a progressively broader assessment than a single open-space avoidance test.

Fig. 7(e) complements the flight scenes by reporting the evolution of UAV linear velocity in all four environments under the common 1.2 m/s limit. The profiles expose both the acceleration required for goal-directed flight and the speed adjustments made during obstacle interactions, rather than

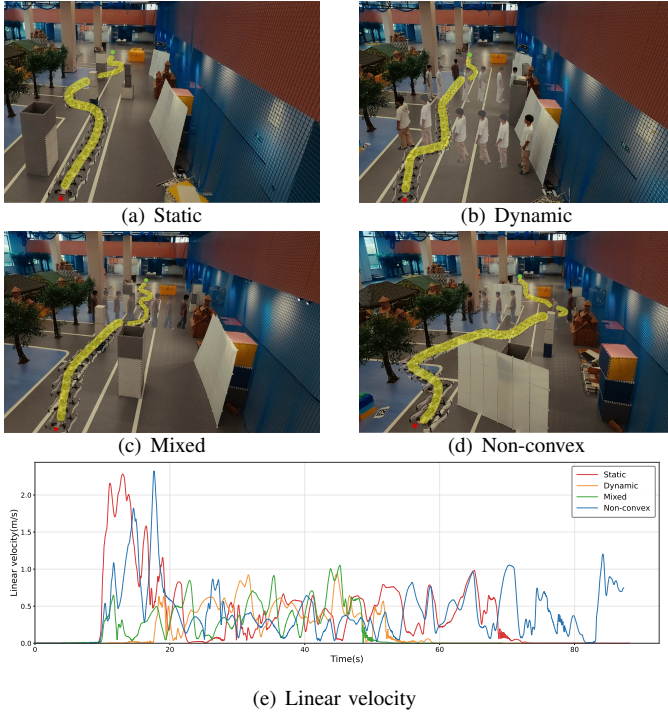


Fig. 7. Zero-shot physical flight tests of ForesightNav in a $28\text{ m} \times 15\text{ m}$ area. (a) Static obstacles. (b) Dynamic obstacles. (c) Mixed static and dynamic obstacles. (d) Large non-convex structures. (e) UAV linear-velocity profiles in the four environments.

evaluating transferability from final paths alone. Together, the representative flights and velocity histories demonstrate closed-loop operation of the transferred policy under real sensing and control effects. These tests provide qualitative evidence of practical deployment feasibility, while the controlled statistical comparisons remain those reported in the simulation study.

V. CONCLUSION

This paper presented ForesightNav, an anticipatory reinforcement-learning framework for UAV navigation in environments with dynamic obstacles and large non-convex structures. A finite-horizon TTC-aware 3D-VO reward introduces relative-motion risk into policy learning, while action-conditioned multi-horizon outcome supervision makes the shared actor-critic representation predictive of near-term collision, trapping, clearance, and progress. Both mechanisms are confined to training, preserving direct policy inference at deployment. Simulation results show improved dynamic-obstacle avoidance and greater robustness to timeouts and local trapping around large structures, and zero-shot flights demonstrate practical deployment feasibility. The formulation remains limited by local observations and a constant-relative-velocity assumption and does not provide a formal safety guarantee. Future work will investigate uncertainty-aware motion prediction and compact temporal representations for longer-horizon navigation around partially observed structures.

REFERENCES

[1] J. Qiu, Q. Liu, J. Qin, D. Cheng, Y. Tian, and Q. Ma, "PE-Planner: A performance-enhanced quadrotor motion planner

for autonomous flight in complex and dynamic environments," *arXiv preprint arXiv:2403.12865*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.12865>

[2] J. Tordesillas, B. T. Lopez, M. Everett, and J. P. How, "FASTER: Fast and safe trajectory planner for navigation in unknown environments," *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 922–938, 2022.

[3] Z. Xu, Y. Xiu, X. Zhan, B. Chen, and K. Shimada, "ViGO: Vision-aided UAV navigation and dynamic obstacle avoidance using gradient-based B-spline trajectory optimization," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2023, pp. 10 699–10 705.

[4] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2017, pp. 285–292.

[5] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, "Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2018, pp. 6252–6259.

[6] Z. Xu, X. Han, H. Shen, H. Jin, and K. Shimada, "NavRL: Learning safe flight in dynamic environments," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3668–3675, 2025.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[8] Z. Xie and P. M. Dames, "DRL-VO: Learning to navigate through crowded dynamic scenes using velocity obstacles," *IEEE Transactions on Robotics*, vol. 39, no. 4, pp. 2700–2719, 2023.

[9] J. Lee, A. Rathod, K. Goel, J. Stecklein, and W. Tabib, "Quadrotor navigation using reinforcement learning with privileged information," *arXiv preprint arXiv:2509.08177*, 2025.

[10] D. Dugas, J. Nieto, R. Siegwart, and J. J. Chung, "NavRep: Unsupervised representations for reinforcement learning of robot navigation in dynamic human environments," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2021, pp. 7829–7835.

[11] C. Chen, S. Hu, P. Nikdel, G. Mori, and M. Savva, "Relational graph learning for crowd navigation," in *Proceedings of the IEEE/RSSJ International Conference on Intelligent Robots and Systems*, 2020, pp. 10 007–10 013, arXiv:1909.13165.

[12] G. Kahn, P. Abbeel, and S. Levine, "BADGR: An autonomous self-supervised learning-based navigation system," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1312–1319, 2021, arXiv:2002.05700.

[13] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.

[14] A. Faust, O. Ramirez, M. Fiser, K. Oslund, A. Francis, J. Davidson, and L. Tapia, "PRM-RL: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2018, pp. 5113–5120.

[15] L. Kästner, X. Zhao, T. Buiyan, J. Li, Z. Shen, J. Lambrecht, and C. Marx, "Connecting deep-reinforcement-learning-based obstacle avoidance with conventional global planners using waypoint generators," in *Proceedings of the IEEE/RSSJ International Conference on Intelligent Robots and Systems*, 2021, pp. 1213–1220.

[16] S. H. Semnani, H. Liu, M. Everett, A. de Ruiter, and J. P. How, "Multi-agent motion planning for dense and dynamic environments via deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3221–3226, 2020, arXiv:2001.06627.

[17] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, "Recovery RL: Safe reinforcement learning with learned recovery zones," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4915–4922, 2021.

[18] N. Kochdumper, H. Krasowski, X. Wang, S. Bak, and M. Althoff, "Provably safe reinforcement learning via action projection using reachability analysis and polynomial zonotopes," *IEEE Open Journal of Control Systems*, vol. 2, pp. 79–92, 2023.

[19] L. He, N. Aouf, J. F. Whidborne, and B. Song, "Deep reinforcement learning based local planner for UAV obstacle avoidance using demonstration data," *arXiv preprint arXiv:2008.02521*, 2020.

[20] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Science Robotics*, vol. 6, no. 59, p. eabg5810, 2021.

[21] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research*, ser. Springer Tracts in Advanced Robotics. Springer, 2011, vol. 70, pp. 3–19.

- [22] R. Han, S. Chen, S. Wang, Z. Zhang, R. Gao, Q. Hao, and J. Pan, "Reinforcement learned distributed multi-robot navigation with reciprocal velocity obstacle shaped rewards," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 5896–5903, 2022.
- [23] R. S. Sutton, J. Modayil, M. Delp, T. Degris, P. M. Pilarski, A. White, and D. Precup, "Horde: A scalable real-time architecture for learning knowledge from unsupervised sensorimotor interaction," in *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2011, pp. 761–768.
- [24] M. Jaderberg, V. Mnih, W. M. Czarnecki, T. Schaul, J. Z. Leibo, D. Silver, and K. Kavukcuoglu, "Reinforcement learning with unsupervised auxiliary tasks," in *International Conference on Learning Representations*, 2017, arXiv:1611.05397.
- [25] M. Schwarzer, A. Anand, R. Goel, R. D. Hjelm, A. Courville, and P. Bachman, "Data-efficient reinforcement learning with self-predictive representations," in *International Conference on Learning Representations*, 2021, arXiv:2007.05929.